CLOUD COMPUTING LABORATORY

Lab Report

Experiment / Assignment 4

Software-Defined Networking (SDN) using Mininet, Open vSwitch and the Ryu Controller

---

Name	Sk Naser Hussain
Roll Number	UG/02/BTCSE/2023/151
Section	B
Course Code	CSE12045
Date of Experiment	15/10/2026
Faculty	Ms. Anusuya Bera

1. Aim

To create a Software-Defined Networking (SDN) environment using Mininet, Open vSwitch and the Ryu controller, and to demonstrate basic forwarding and filtering of network traffic using OpenFlow flow rules.

2. Theory

2.1 Software-Defined Networking (SDN)

Software-Defined Networking is a network architecture in which the control plane (the logic that decides how traffic is routed) is separated from the data plane (the hardware or software that actually forwards packets). A centralized software controller has a global view of the network and programs the forwarding behaviour of the switches, which makes the network flexible, programmable and easier to manage.

2.2 OpenFlow

OpenFlow is the standard southbound protocol used by an SDN controller to communicate with switches. Each switch holds a flow table; every flow entry has match fields (for example input port or MAC address), a priority, and actions (for example output to a port, drop, or send to the controller). When a packet matches no entry, the switch sends it to the controller using the table-miss entry, and the controller can then install a new flow rule.

2.3 Tools Used

- **Mininet:** A network emulator that creates virtual hosts, switches and links on a single machine, so that SDN topologies can be tested without physical hardware.
- **Open vSwitch (OVS):** A production-quality, software-based virtual switch that supports OpenFlow and acts as the data plane in this experiment.
- **Ryu Controller:** A Python-based SDN controller framework. The application ryu.app.simple_switch_13 implements a learning switch using OpenFlow 1.3.
- **ovs-ofctl:** A utility used to inspect and modify the flow tables of an OpenFlow switch.

3. Requirements

- **Operating system:** Ubuntu 24.04 (noble) running in Oracle VirtualBox
- **Software:** Mininet 2.3.0, Open vSwitch 3.3.9, Ryu 4.34
- **Language / tools:** Python 3 with pip and venv (virtual environment) support
- **Network:** Internet connection for installing packages

4. Procedure

Step 1: Update Ubuntu

The package index was refreshed and installed packages were upgraded so that the system is up to date before installing new software.

```
sudo apt update && sudo apt upgrade -y
```

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo apt update && sudo apt upgr
ade -y
Hit:1 http://in.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,15
7 kB]
Get:6 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,4
25 kB]
```

Figure 1: Updating and upgrading the Ubuntu system

Step 2: Install Mininet

Mininet was installed to create virtual hosts, switches and links. The output shows that Mininet 2.3.0-1.1 is already the newest version available.

```
sudo apt install mininet -y
```

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo apt install mininet -y
[sudo] password for sk-naser-hussain:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
mininet is already the newest version (2.3.0-1.1).
The following package was automatically installed and is no longer required:
  libfwupd2
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

Figure 2: Installation of Mininet

Step 3: Install Open vSwitch

Open vSwitch, the software-based SDN switch, was installed. Version 3.3.9 is already the newest available version.

```
sudo apt install openvswitch-switch -y
```

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo apt install openvswitch-swi
tch -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openvswitch-switch is already the newest version (3.3.9-0ubuntu0.24.04.1).
openvswitch-switch set to manually installed.
The following package was automatically installed and is no longer required:
  libfwupd2
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

Figure 3: Installation of Open vSwitch

Step 4: Check Open vSwitch

The service status was checked to confirm that Open vSwitch is running. The service is shown as loaded and active.

```
sudo systemctl status openvswitch-switch
```

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo systemctl status openvswitc
h-switch
● openvswitch-switch.service - Open vSwitch
     Loaded: loaded (/usr/lib/systemd/system/openvswitch-switch.service; enable>
     Active: active (exited) since Fri 2026-11-06 21:02:36 IST; 53min ago
   Main PID: 747 (code=exited, status=0/SUCCESS)
        CPU: 50ms

Nov 06 21:02:34 sk-naser-hussain-VirtualBox systemd[1]: Starting openvswitch-sw>
Nov 06 21:02:36 sk-naser-hussain-VirtualBox systemd[1]: Finished openvswitch-sw>
lines 1-8/8 (END)
```

Figure 4: Open vSwitch service status

Step 5: Test Mininet

A default Mininet network was started to verify the installation. It creates two hosts (h1, h2) and one switch (s1).

```
sudo mn
```

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo mn
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller

*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>
```

Figure 5: Starting the default Mininet network

The nodes command lists the virtual hosts and the switch:

```
mininet> nodes
```

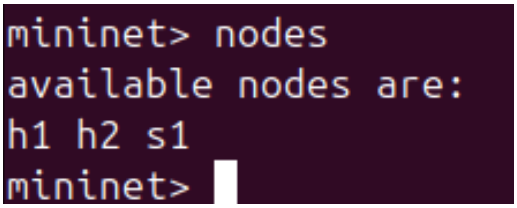


Figure 6: Nodes available in the default topology (h1, h2, s1)

The pingall command tests connectivity between all hosts. Both pings succeeded with 0% packet loss:

```
mininet> pingall
```

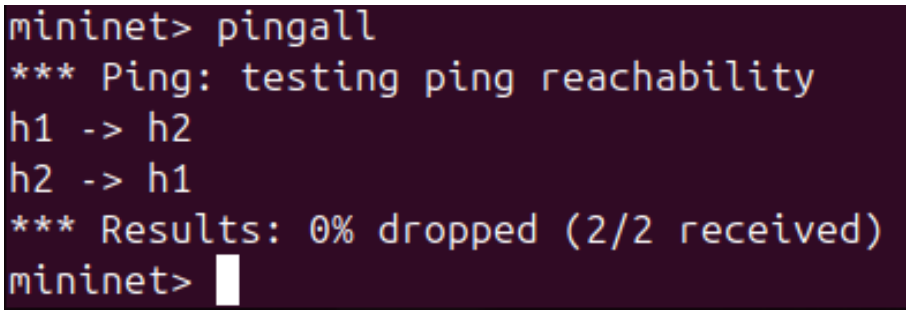


Figure 7: pingall result in the default topology: 0% dropped (2/2 received)

The Mininet CLI was then closed and the network was stopped:

```
mininet> exit
```

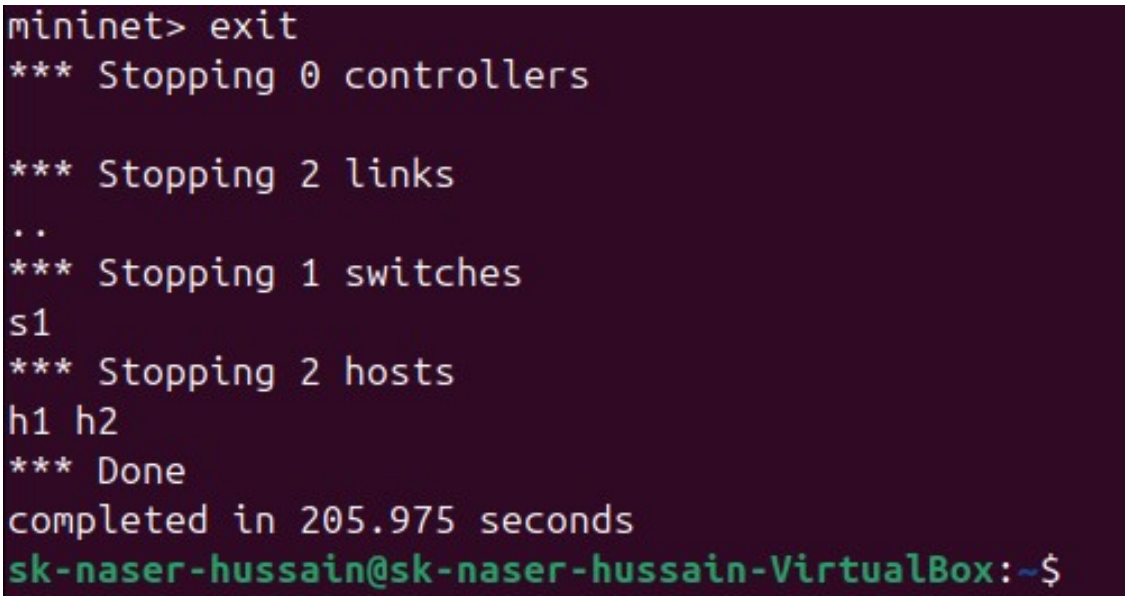


Figure 8: Exiting Mininet

Step 6: Install Python and Virtual Environment

Python pip and venv support were installed, along with the required build tools, so that Ryu can be installed in an isolated environment.

```
sudo apt install python3-pip python3-venv -y
```

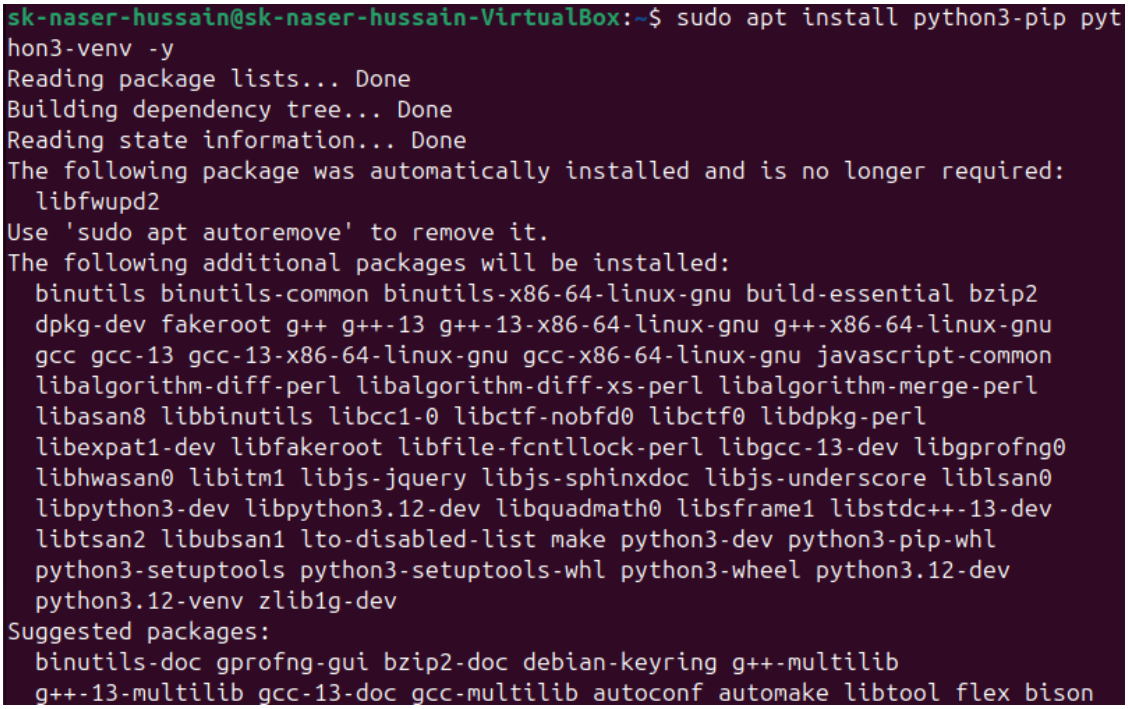


Figure 9: Installation of python3-pip and python3-venv

Step 7: Create Ryu Environment

A Python virtual environment was created for Ryu and then activated. The shell prompt is prefixed with the environment name once it is active.

```
python3 -m venv ryu-env
```

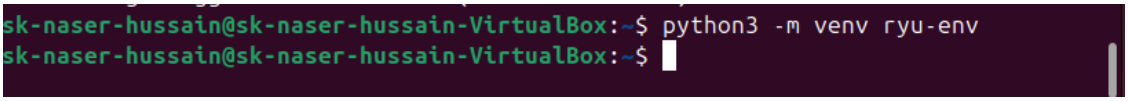


Figure 10: Creating the virtual environment

```
source ryu-env/bin/activate
```

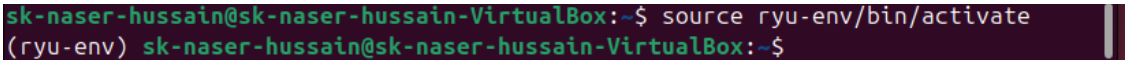


Figure 11: Activating the virtual environment

Step 8: Install Ryu

The Ryu SDN controller was installed inside the virtual environment using pip.

```
pip install ryu
```

Step 9: Test Ryu Installation

The installation was verified by checking the Ryu version. The first attempt failed with an ImportError caused by an incompatibility between Ryu and the installed Eventlet library under the default Python version.

```
ryu-manager --version
```

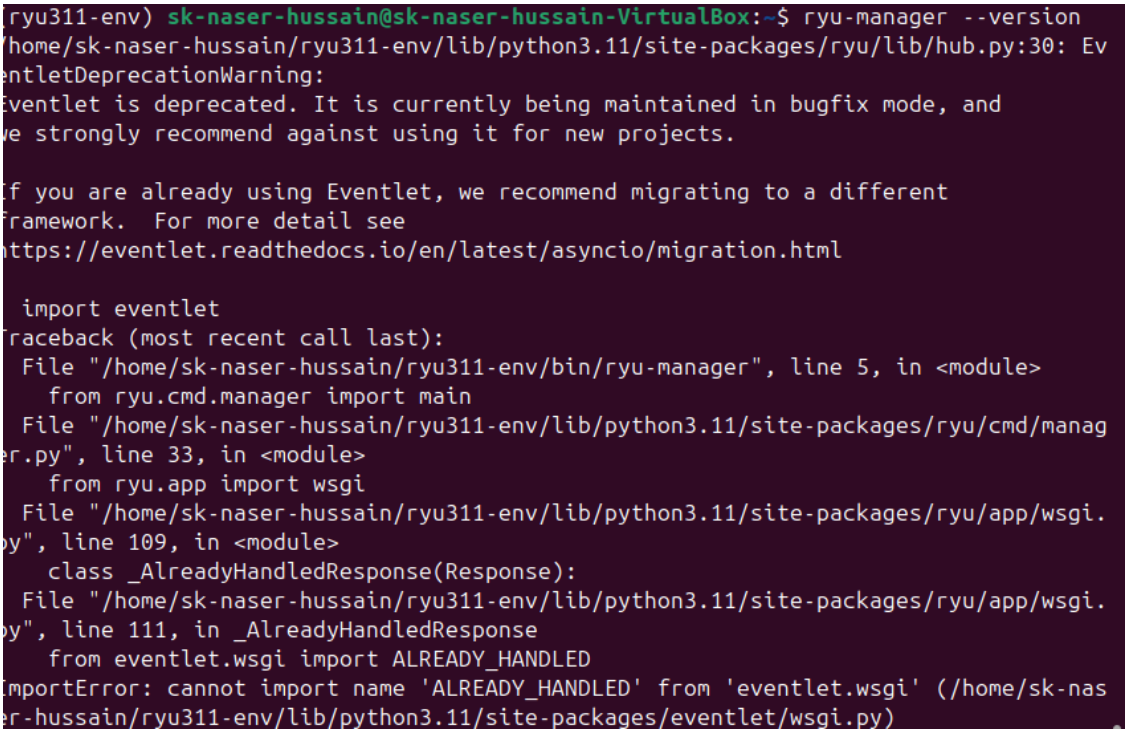


Figure 12: ImportError raised by ryu-manager (Eventlet incompatibility)

The problem was resolved by using a separate Python 3.11 virtual environment (ryu311-env) for Ryu. In this environment the command works and reports Ryu version 4.34:

```
ryu-manager --version
```

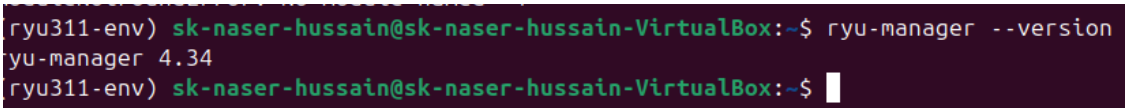


Figure 13: ryu-manager --version displays Ryu 4.34

Step 10: Start Ryu Controller

The Ryu controller was started with the basic OpenFlow 1.3 learning-switch application. This terminal is kept open while the experiment runs.

```
ryu-manager ryu.app.simple_switch_13
```

```
(ryu311-env) sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ ryu-manager ryu.app.sim
ple_switch_13
loading app ryu.app.simple_switch_13
loading app ryu.controller.ofp_handler
instantiating app ryu.app.simple_switch_13 of SimpleSwitch13
instantiating app ryu.controller.ofp_handler of OFPHandler
```

Figure 14: Ryu controller running simple_switch_13

Step 11: Create Mininet SDN Topology

In a new terminal, a Mininet topology with three hosts connected to one Open vSwitch was created, using the remote Ryu controller (127.0.0.1, port 6653).

```
sudo mn --topo single,3 --controller=remote --switch ovsk
```

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo mn --topo single,3 --contro
ller=remote --switch ovsk
[sudo] password for sk-naser-hussain:
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>
```

Figure 15: Mininet topology with 3 hosts, 1 switch and a remote controller

Step 12: Check Nodes

The nodes command lists the controller, the three hosts and the switch.

```
mininet> nodes
```

```
mininet> nodes
available nodes are:
c0 h1 h2 h3 s1
mininet>
```

Figure 16: Nodes in the SDN topology (c0, h1, h2, h3, s1)

Step 13: Check Connections

The net command shows how each host is linked to the switch: h1 to s1-eth1, h2 to s1-eth2 and h3 to s1-eth3.

```
mininet> net
```

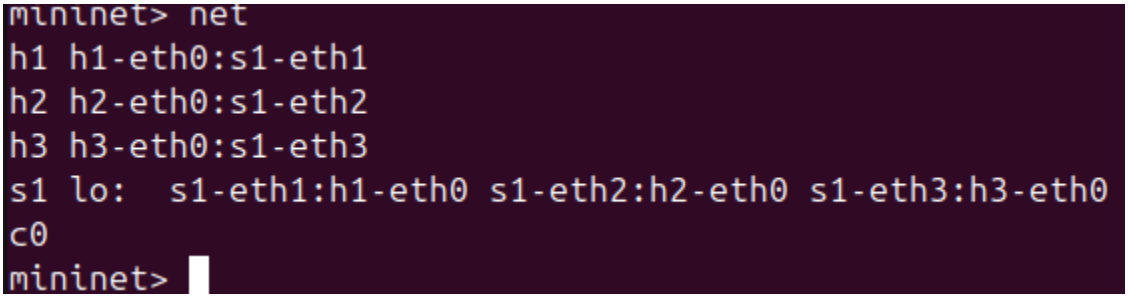


Figure 17: Links between the hosts and the switch

Step 14: Test Connectivity

The pingall command verifies communication between all hosts. All 6 pings were received with 0% packet loss, which shows that the Ryu controller is forwarding traffic correctly.

```
mininet> pingall
```

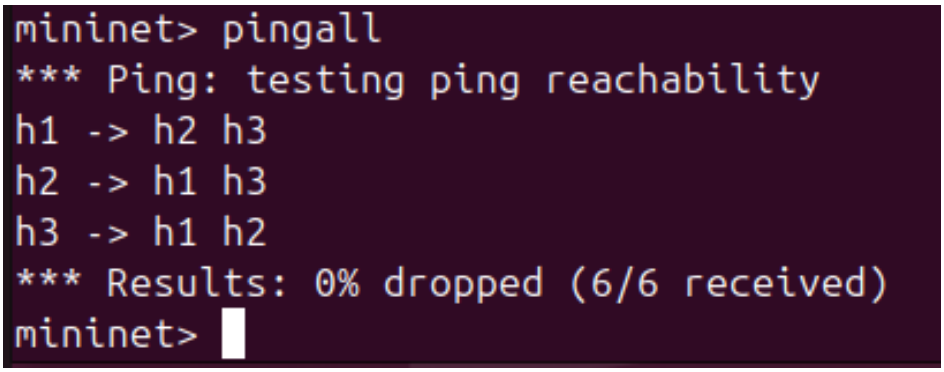


Figure 18: pingall result: 0% dropped (6/6 received)

```
mininet> h1 ping h2
```

Step 15: View Flow Rules

In another terminal, the flow table of switch s1 was displayed. The learning switch application installed flow entries for each pair of communicating hosts, plus a table-miss entry that sends unmatched packets to the controller.

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

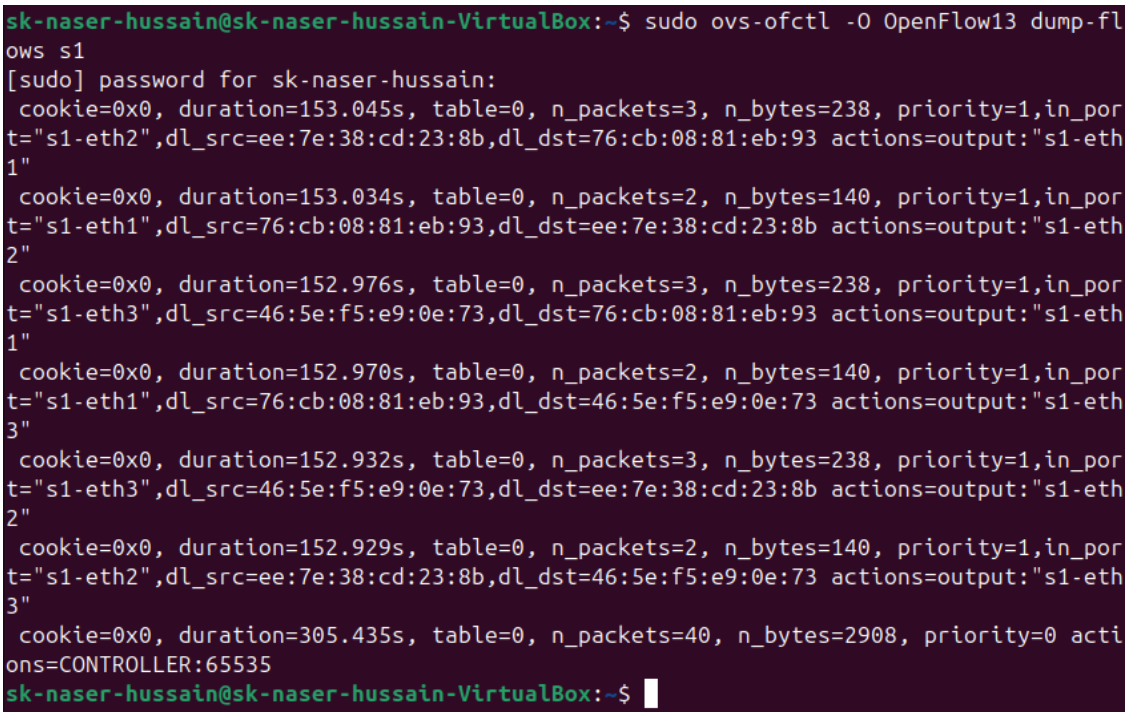


Figure 19: Flow rules installed in s1

Step 16: Add a Simple Filtering Rule

A manual flow rule with priority 100 was added to drop every packet that enters switch s1 through port 1 (the port connected to h1). Its priority is higher than that of the learned forwarding rules (priority 1), so it takes precedence.

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1
"priority=100,in_port=1,actions=drop"
```

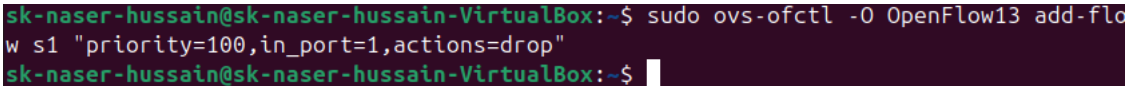


Figure 20: Adding the drop rule for port 1 of s1

Step 17: Test the Filtering Rule

With the drop rule installed, traffic from h1 (which enters s1 on port 1) is dropped, so the ping from h1 to h2 is expected to fail.

```
mininet> h1 ping h2
```

Step 18: Remove Flow Rules

The manually added flow rules were removed from the switch.

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1
```

Step 19: Clean Mininet

Mininet interfaces and leftover processes were cleaned up.

```
sudo mn -c
```

5. Observations

- **Basic test:** The default Mininet network (2 hosts, no remote controller) showed 0% packet loss, confirming that Mininet and Open vSwitch work correctly.
- **Controller connection:** The Ryu controller started with simple_switch_13 and the 3-host Mininet topology connected to it at 127.0.0.1:6653.
- **Connectivity:** pingall in the SDN topology reported 0% dropped (6/6 received), so the controller forwards traffic between all hosts.
- **Flow table:** After the pings, the switch contained six learned flow entries (priority 1), one for each direction between each pair of hosts, and a table-miss entry (priority 0) that sends unmatched packets to the controller:

#	In port	Source MAC	Destination MAC	Action	Packets / bytes
1	s1-eth2	ee:7e:38:cd:23:8b	76:cb:08:81:eb:93	out s1-eth1	3 / 238
2	s1-eth1	76:cb:08:81:eb:93	ee:7e:38:cd:23:8b	out s1-eth2	2 / 140
3	s1-eth3	46:5e:f5:e9:0e:73	76:cb:08:81:eb:93	out s1-eth1	3 / 238
4	s1-eth1	76:cb:08:81:eb:93	46:5e:f5:e9:0e:73	out s1-eth3	2 / 140
5	s1-eth3	46:5e:f5:e9:0e:73	ee:7e:38:cd:23:8b	out s1-eth2	3 / 238
6	s1-eth2	ee:7e:38:cd:23:8b	46:5e:f5:e9:0e:73	out s1-eth3	2 / 140
T	any (priority 0)	any	any	CONTROLLER	40 / 2908

Table 1: Flow entries observed on switch s1 (T = table-miss entry)

- **Filtering rule:** The rule "priority=100,in_port=1,actions=drop" was added to s1. As it has a higher priority than the learned rules, packets arriving on port 1 (from h1) match it first and are dropped.

6. Result

The SDN environment was successfully created using Mininet, Open vSwitch, and the Ryu Controller, and basic forwarding and filtering of network traffic was demonstrated.

7. Conclusion

In this experiment, a virtual SDN network was built with Mininet and Open vSwitch and controlled by a remote Ryu controller running the OpenFlow 1.3 learning-switch application. The controller dynamically installed flow rules in the switch so that all hosts could communicate. The flow table was inspected with ovs-ofctl, and a high-priority drop rule was added manually to filter traffic from a single port. This shows how the separation of the control plane from the data plane allows network behaviour to be programmed and changed centrally and in software.